SYSTEM PROGRAMMING / MIDTERM

UNIX

期中题目解析与答案

2025 / 2023

题目 / 解析 / 建议答案 / 教材定位　　36 + 6

01	2025 年基础题	17 道	02
02	2023 年基础题	19 道	10
03	补拟代码题	6 道	18

使用说明

- 2025 年部分保留回忆稿中的 17 道基础题及原题号；原第 5、10 题缺失，不补造原题。
- 2023 年部分保留现有材料中的 19 道基础题。两年的重复题分别给出完整解析，不用“同上”代替。
- 现有 2023 年材料只包含这 19 道基础题，不能据此断言当年没有其他题。2025 年材料只记下五类代码题考点，没有完整原卷代码；后面的 6 道代码练习均为补拟，不是原题复原。
- 教材定位已按当前英文第 3 版 PDF 正文逐项重核，同时列出书上正文页码和 PDF 页序。PDF 页序从文件第一张算第 1 页，换算关系为：PDF 页序 = 书上正文页码 + 34；不要套用往年回忆稿中的其他版本页码。
- 权限题默认讨论普通本地 UNIX 文件权限模型，不考虑 ACL、只读挂载等额外限制；涉及粘滞位、特殊身份的地方另外说明。
- “建议答案”是依据教材和接口语义整理的复习答案，不是教师发布的官方答案。措辞有歧义的题会保留题干，并说明判断所需的限定。

CHAPTER 01

2025 年基础题

17 道记录题目 · 保留回忆稿原题号

01 文件所有者与组所有者

2025-01

题目 每一个文件都有一个所有者和组所有者，所有者由 stat 结构中的 st_uid 成员表示，组所有者则用 st_gid 表示。

解析 stat 返回文件的属性。st_uid 是文件所有者的用户 ID，st_gid 是文件所属组的组 ID，保存的都是数字标识，不是用户名和组名。

这里说的是文件的属性，不是当前访问文件的进程身份。进程的实际用户 ID、有效用户 ID 与文件的 st_uid 是不同概念。

建议答案 **是。st_uid 表示文件所有者的用户 ID，st_gid 表示文件所属组的组 ID。**

教材定位
- §4.4，第 98 页（PDF 第 132 页）末段：题干关于文件所有者、组所有者及 st_uid、st_gid 的直接原文。
- §4.2，第 94 页（PDF 第 128 页）：stat 结构定义及两个成员的含义。

02 复制文件内容

2025-02

题目 UNIX 将一个文件的内容复制到另一个文件中，使用什么方法?

解析 若问命令，使用 cp，例如 cp source.txt destination.txt。

若问系统编程方法，就打开源文件，创建或打开目标文件，循环调用 read 读取源数据、write 写入目标文件，最后关闭文件。

每轮复制都应以 read 返回的实际字节数作为有效数据长度，不能不加区分地写出整个缓冲区。完整实现还需要处理短写。

回忆题没有说明问的是命令还是接口，答题时把两层都写上更稳妥。

建议答案 **使用 cp 命令。程序实现可通过 open/creat 打开或创建文件，再循环调用 read 和 write 复制内容，最后调用 close 关闭文件。**

教材定位
- §1.5，第 9 页（PDF 第 43 页），图 1.4：复制循环及 read 返回值的用法。
- §3.3，第 62 页（PDF 第 96 页）：open；§3.4、§3.5，第 66 页（PDF 第 100 页）：creat、close。
- §3.7，第 71 页（PDF 第 105 页）：read；§3.8，第 72 页（PDF 第 106 页）：write。cp 命令另见文末 GNU Coreutils 文档，不把图 1.4 当作 cp 命令的说明。

03 可执行文件的粘住位

2025-03

题目 如果一个可执行程序被设置了“粘住位”，那么在该程序第一次被执行并结束时，其程序正文部分的一个副本仍被保存在交换区，这使得下次执行时能够较快地将其装入内存。

解析 这是 sticky bit 的历史用途。早期 UNIX 把程序正文，即机器指令部分，保留在交换区，以加快下一次装入。

不是把整个进程的变量和运行状态都保存下来，也不是说现代 Linux 仍使用这个位为普通可执行文件加速。

现代系统中更常见的是目录上的粘滞位，用于给删除、改名增加所有权等限制，例如 /tmp。两种用途不能混淆。

建议答案 **按教材介绍的历史含义，是。程序正文副本在进程结束后保留在交换区，以加快后续装入。**

教材定位 §4.10，第 108-109 页（PDF 第 142-143 页）：历史用途从第 108 页末段开始，解释续至第 109 页；目录粘滞位的删除、改名限制在第 109 页。

04 多个 UID 为 0 的账号

2025-04

题目 一个系统中可以有多个 ID 为 0 的用户。

解析 这里的 ID 指 UID。登录名与数字 UID 不是同一概念，不同登录名可以映射到相同的 UID，包括 UID 0。

在内核的身份和权限判断中，这些账号具有相同的数字用户身份。通常为每个账号分配不同 UID，是管理约定，不是绝对不能重复。

因此应准确理解为“可以有多个 UID 为 0 的登录账号”，不是说内核把同一个数字 0 识别为多个不同的用户身份。实际管理中不建议这样配置。

建议答案 **是。不同登录名可以具有相同的 UID；多个登录账号可以具有 UID 0，并具有超级用户身份。**

教材定位 §1.8，第 16 页（PDF 第 50 页）“User ID”：UID 通常唯一分配，UID 0 表示超级用户。注意：本节并未直接写“可以有多个 UID 0 账号”；重复 UID 的直接依据见文末 useradd 手册，不把这条结论标成教材原话。

06 UNIX 时间的数据类型

2025-06

题目 UNIX 内核提供的时间以数据类型 time_t 表示。

解析 这句话省略了时间种类。教材 §6.10 所说的基本时间服务，是从 1970 年 1 月 1 日 00:00:00 UTC 起计算的日历时间，其秒数用 time_t 表示。

教材 §1.10 还介绍了进程 CPU 时间，传统上以时钟滴答计数，用 clock_t 表示。不能把所有时间都概括成 time_t。

建议答案 **按教材“基本时间服务”的日历时间语境，答“是”，这里的时间用 time_t 表示。若题目明确说“所有种类的时间都用 time_t”，则应答“否”；进程 CPU 时间的传统计数用 clock_t 表示。**

教材定位
- §6.10，第 189 页（PDF 第 223 页）首段：内核基本时间服务、Epoch 和 time_t，是判断题的直接对应段落。
- §1.10，第 20 页（PDF 第 54 页）：日历时间 time_t 与进程时间 clock_t 的区分。

07 目录的读权限与执行权限

2025-07

题目 对于目录的读权限和执行权限，意义相同吗？为什么？

解析 目录的读权限 r 允许读取目录项、列出文件名。目录的执行权限 x 也叫搜索权限，允许按名字查找目录中的文件，并通过该目录解析路径。

只有 x 而没有 r 时，可能不能列出里面有哪些文件，但知道名字时仍可能访问该文件，前提是其他相关权限允许。

只有 r 而没有 x 时，可以取得名字列表，却通常不能获取这些名字对应文件的属性或打开它们。

建议答案 **不相同。目录的读权限允许列出文件名；执行权限允许搜索目录、按名字访问其中的文件。**

教材定位 §4.5，第 100 页（PDF 第 134 页）：第一条规则中直接对比目录的 read permission 与 execute/search permission。

08 创建新文件的目录权限

2025-08

题目 为了在一个目录中创建一个新文件，只需对该目录具有写权限。

解析 创建新文件需要修改父目录的目录项，因此需要 w；还要在该目录中搜索和处理新文件名，因此需要 x。

目录的 r 不是创建新文件的必要条件。没有列出目录内容的权限，不等于不能创建一个已知名字的新文件。

建议答案 **否。创建新文件通常需要对父目录同时具有写权限和执行权限，即 w+x。**

教材定位 §4.5，第 100 页（PDF 第 134 页）：靠近页末的创建文件规则，明确要求目录的 write permission 和 execute permission。

09 删除文件需要什么权限

2025-09

题目 为了删除一个现有文件，必须对包含该文件的目录具有写权限和执行权限，而对该文件本身则不需要任何权限。

解析 删除文件名是在父目录中删除对应的目录项，不是先修改文件的数据内容。因此通常检查父目录的 w+x，而不要求目标文件本身可读、可写或可执行。

如果父目录设置了粘滞位，还会增加限制，通常要求操作者是文件所有者、目录所有者或具有相应特权。

建议答案 **按普通文件权限规则，是。需要父目录的写权限和执行权限，不要求目标文件本身具有读、写或执行权限；粘滞位等机制可能增加限制。**

教材定位
- §4.5，第 100 页（PDF 第 134 页）：靠近页末的删除文件规则，检查所在目录的写、执行权限，不要求目标文件本身可读、可写。
- §4.10，第 109 页（PDF 第 143 页）：目录设置粘滞位时的附加限制。

11 不带缓冲的 I/O

2025-11

题目 什么是不带缓冲的 I/O?

解析 按教材第三章的术语，read、write 等操作文件描述符的接口属于不带缓冲 I/O。每次 read 或 write 都请求内核服务，不经过标准 I/O 库的流缓冲。

这里不是说内核没有缓存，也不是说每次调用都会实际访问磁盘。程序自己使用一个 char 数组作为数据缓冲，也不等于使用了标准 I/O 库的流缓冲。

另一个相关概念是第五章中“标准 I/O 流的不带缓冲模式”，指该流不在 C 库中积攒字符。若题目明确在问标准 I/O 的三种缓冲，应按第五章的含义回答。

建议答案 **不带缓冲 I/O 是通过 read、write 等接口进行的 I/O，每次调用都请求内核服务，不使用标准 I/O 库的流缓冲，但内核仍可能使用缓存。**

教材定位
- §3.1，第 61 页（PDF 第 95 页）：第三章“不带缓冲 I/O”的定义，每次 read 或 write 都调用内核中的系统调用。
- §5.4，第 146 页（PDF 第 180 页）第 3 项“Unbuffered”：标准 I/O 库不缓冲字符，这是另一种语境。
- §3.13，第 81 页（PDF 第 115 页）首段：内核的 buffer cache/page cache，说明“不带缓冲”不等于内核没有缓存。

12 时钟时间

2025-12

题目 UNIX 中，时钟时间是什么?

解析 时钟时间也叫墙上时钟时间，即 wall clock time，是程序从开始运行到结束实际经过的时间，包含执行、等待 I/O、等待调度、睡眠等经过的时间。

例如程序经过 10 秒结束，但只消耗了 2 秒 CPU 时间，则墙钟时间是 10 秒，不是 2 秒。time 命令输出的 real 对应墙钟时间，user 和 sys 分别对应用户 CPU 时间与系统 CPU 时间。

建议答案 **时钟时间又称墙上时钟时间，是进程运行所经过的实际时间，包括执行和等待的时间，其值会受到系统中其他进程活动等因素的影响。**

教材定位 §1.10，第 20 页（PDF 第 54 页）：三种进程执行时间及 wall clock time 的定义；同页 time 命令示例列出 real、user、sys。

13 修改密码涉及的文件

2025-13

题目 UNIX 中修改密码需要访问到哪些文件?

解析 按常见的本地 shadow 口令机制，/etc/passwd 保存登录名、UID、GID、主目录、登录 Shell 等用户基本信息；口令哈希及口令老化信息保存在 /etc/shadow。

修改密码通常需要查找账号信息，并更新受保护的口令记录。不能说 /etc/shadow 存储明文口令，也不能说每次修改密码一定同时写入这两个文件。

本题不讨论网络账号、其他认证后端等情形。

建议答案 **主要涉及 /etc/passwd 和 /etc/shadow。前者保存用户基本信息；在使用 shadow 的系统中，后者保存口令哈希及老化信息，修改密码主要更新相应的 shadow 口令记录。**

教材定位
- §4.4，第 99 页（PDF 第 133 页）：passwd 程序修改口令文件，直接列出 /etc/passwd 和 /etc/shadow。
- §6.2，第 178 页（PDF 第 212 页），图 6.1 及其下方说明：/etc/passwd 的字段、示例及口令占位符。
- §6.3，第 181 页（PDF 第 215 页），图 6.3 及正文：shadow 口令文件、单向处理后的口令记录与老化字段。

14 绝对路径与相对路径

2025-14

题目 绝对路径和相对路径有什么区别?

解析 以斜线 / 开头的是绝对路径，从根目录开始解析。不以 / 开头的是相对路径，通常从进程当前工作目录开始解析。

例如 /home/student/a.txt 是绝对路径，a.txt、./a.txt、../a.txt 都是相对路径。相对路径不一定以一个点开头。

建议答案 **绝对路径以 / 开头，从根目录开始解析；相对路径不以 / 开头，通常以进程当前工作目录为解析起点。**

教材定位
- §1.4，第 5 页（PDF 第 39 页）“Pathname”：绝对路径以 / 开头，相对路径的定义。
- §1.4，第 8 页（PDF 第 42 页）“Working Directory”：相对路径与进程当前工作目录的关系。

15 点与点点

2025-15

题目 .（称为点）和 ..（称为点一点）分别指什么?

解析 在一个目录中，. 指该目录自身，.. 指它的父目录。根目录没有更高的父目录，所以根目录中的 .. 仍指根目录。

在递归遍历目录时通常要跳过这两个目录项，否则可能回到自身或父目录，造成重复遍历甚至循环。

建议答案 **. 表示当前目录自身，.. 表示父目录；在根目录中，.. 与 . 都指根目录。**

教材定位 §1.4，第 4 页（PDF 第 38 页）最末段：.、.. 的含义，以及根目录中的 .. 与 . 相同。

16 有访问权限就能读取目录吗

2025-16

题目 对某个目录具有访问权限的任意用户都可以读取该目录。

解析 题干中的“访问权限”不够明确。教材 §4.22 的意思是：具有读取该目录权限的用户可以读取目录，而不是说任何形式的访问许可都足够。

只有 x 搜索权限，不代表具有 r 读权限，也就不能据此断言可以列出目录项。

建议答案 **按教材这里的语境，即“具有读目录的权限”，答“是”。如果“访问权限”只指 x 搜索权限，则这个说法不成立。**

教材定位
- §4.22，第 130 页（PDF 第 164 页）首段：原文限定为有权限读取该目录，不是仅有搜索权限。
- §4.5，第 100 页（PDF 第 134 页）：目录的读权限与执行/搜索权限并不相同。

17 ls 所需的目录权限

2025-17

题目 若我们想在某个目录下执行 ls 命令，需要对这个目录有什么权限?

解析 如果题意是“列出目录中的文件名”，需要目录的 r 权限。路径中沿途目录仍需要 x 搜索权限。

如果还要先通过 cd 进入该目录，或者用 ls -l 获取目录内各项的属性，通常还需要该目录的 x 权限。列名字、进入目录、取得属性是不同操作。

建议答案 **仅列出目录中的文件名需要读权限 r；若还要进入目录或取得内部文件的属性，通常还需要执行权限 x。**

教材定位
- §4.5，第 100 页（PDF 第 134 页）：读权限允许列出文件名；执行/搜索权限用于路径查找。
- §4.22，第 130 页（PDF 第 164 页）：读目录所需的权限。进入目录另见 §4.23，第 135 页（PDF 第 169 页）的 chdir。

18 Shell 的定义

2025-18

题目 Shell 是什么?

解析 Shell 是用户与系统交互的一种程序，读取并解释命令，然后执行相应操作。输入既可以来自终端，也可以来自脚本文件。

Shell 不是内核。对于外部命令，它通常启动相应程序；有些操作由 Shell 自身的内建命令完成，例如 cd。

建议答案 **Shell 是命令行解释器，读取、解释并执行用户命令；命令可以来自终端，也可以来自 Shell 脚本文件。**

教材定位
- §1.3，第 3 页（PDF 第 37 页）“Shells”首段：Shell 的定义，以及终端、脚本两种输入来源。
- §4.23，第 135-136 页（PDF 第 169-170 页），图 4.23 及其后说明：外部程序不能改变父 Shell 的工作目录，cd 因而是内建命令。

19 空洞与磁盘存储区

2025-19

题目 空洞文件需要在磁盘上占用存储区。

解析 这道回忆题把“空洞文件”和“文件中的空洞部分”混在一起。教材的准确意思是：文件中的空洞部分不要求有对应的磁盘数据块。

整个空洞文件仍可能占用元数据和已写入部分的数据块；某些文件系统中，位于同一个已分配块内的小空洞也未必节省实际磁盘空间。

因此不能把“空洞不要求实际分配数据块”说成“整个文件完全不占磁盘”。

建议答案 **若本题考的是“文件中的空洞是否必须占用实际存储区”，答“否”：空洞部分不要求分配对应的磁盘数据块，读取时返回值为 0 的字节。文件的已写入部分和元数据仍可能占用空间。**

若严格按“整个空洞文件是否占用任何空间”理解，则不能笼统答“不占用”；回忆题措辞需要结合题意判断。

教材定位

- §3.6，第 68 页（PDF 第 102 页）：lseek 不执行 I/O、越过文件尾再写入可形成空洞、未写入位置读回 0，以及空洞不要求有磁盘存储区的直接原文。
- §3.6，第 68-69 页（PDF 第 102-103 页），图 3.2 及输出：空洞文件示例。
- §4.12，第 111-112 页（PDF 第 145-146 页）“Holes in a File”：逻辑长度与实际磁盘占用的对比。

CHAPTER 02

2023 年基础题

19 道记录题目 · 重复考点分别完整解析

现有材料说明：句号结尾的题回答“是”或“否”；问号结尾的题简要解释。以下保留原题顺序。

01 文件所有者与组所有者

2023-01

题目 每一个文件都有一个所有者和组所有者，所有者由 stat 结构中的 st_uid 成员表示，组所有者则用 st_gid 表示。

解析 st_uid 和 st_gid 都属于文件属性，分别记录文件所有者的用户 ID 和文件所属组的组 ID，而不是当前进程的用户身份。数字 ID 可以经用户、组信息映射为相应名称。

建议答案 **是。st_uid 表示文件所有者的用户 ID，st_gid 表示文件所属组的组 ID。**

教材定位
- §4.4，第 98 页（PDF 第 132 页）末段：题干关于文件所有者、组所有者及 st_uid、st_gid 的直接原文。
- §4.2，第 94 页（PDF 第 128 页）：stat 结构定义及两个成员的含义。

02 流的定向

2023-02

题目 流的定向决定了一个所读写的字符是单字节还是多字节的。

解析 这与教材 §5.2 的表述一致。更准确地说，流有字节定向和宽字符定向，分别对应使用字节 I/O 接口和宽字符 I/O 接口。

流最初未定向，第一次相应 I/O 操作可以确定其定向。fwide 可以查询定向，也可以尝试对尚未定向的流设定方向，但不能把已经定向的流直接改成另一种方向。

不要把这句话理解为“字节定向的流只能处理 ASCII”。例如 UTF-8 数据也可以作为字节序列进行读写，流定向不是字符编码的选择开关。

建议答案 **按教材表述，是。流的定向决定使用字节 I/O 还是宽字符 I/O；实际应区分字节定向与宽字符定向，而不是简单地根据字符编码占几个字节来判断。**

教材定位 §5.2，第 144 页（PDF 第 178 页）首段及 fwide 说明：流的定向、首次 I/O 定向，以及已定向的流不能用 fwide 直接改变方向。第 143 页只是本节开头，具体定向论述在第 144 页。

03 可执行文件的粘住位

2023-03

题目 如果一个可执行程序被设置了“粘住位”，那么在该程序第一次被执行并结束时，其程序正文部分的一个副本仍被保存在交换区，这使得下次执行时能够较快地将其装入内存。

解析 这是早期 UNIX 的历史机制，保留的是程序正文即机器指令部分，不是整个进程的全部运行状态。

现代系统中目录上的 sticky bit 主要用于限制删除和改名，不能把历史机制直接当成现代 Linux 普通可执行文件的行为。

建议答案 **按教材介绍的历史含义，是。粘住位可以使程序正文副本保留在交换区，以加快后续装入。**

教材定位 §4.10，第 108-109 页（PDF 第 142-143 页）：历史用途从第 108 页末段开始，解释续至第 109 页；目录粘滞位的删除、改名限制在第 109 页。

04 多个 UID 为 0 的账号

2023-04

题目 一个系统中可以有多个 ID 为 0 的用户。

解析 不同登录名可以配置为相同的 UID，包括 0。内核根据数字身份进行相关判断，这些账号具有相同的超级用户身份。

原整理稿写“不可以”，并说明没有找到教材原话。这个解释把“通常唯一分配”误当成“系统绝对禁止重复”，本稿予以纠正。

建议答案 **是。可以存在多个 UID 为 0 的登录账号；它们在内核看来具有相同的数字用户身份。这种配置不建议用于实际管理。**

教材定位 §1.8，第 16 页（PDF 第 50 页）“User ID”：UID 通常唯一分配，UID 0 表示超级用户。注意：本节并未直接写“可以有多个 UID 0 账号”；重复 UID 的直接依据见文末 useradd 手册，不把这条结论标成教材原话。

05 标准 I/O 的三种缓冲

2023-05

题目 标准 I/O 库提供了哪几种类型的缓冲?

解析 全缓冲通常在库缓冲区满时提交数据，也可以通过显式刷新等方式提前提交。

行缓冲通常在遇到换行时提交输出；缓冲区满和其他规定的刷新条件也可能触发提交，不能说“只有换行才输出”。

不带缓冲模式不在标准 I/O 库中积攒字符，但不代表内核没有缓存。常见实现中，连接终端的 stdout 为行缓冲，重定向到普通文件时为全缓冲；stderr 通常不带缓冲。

建议答案 **全缓冲、行缓冲和不带缓冲。全缓冲通常在缓冲区满时提交，行缓冲通常在遇到换行时提交，不带缓冲模式不在标准 I/O 库中积攒字符。**

教材定位 §5.4，第 145-146 页（PDF 第 179-180 页）：第 145 页列出全缓冲、行缓冲；第 146 页列出不带缓冲及标准流的常见默认模式。若要查设置方式，再看第 147 页（PDF 第 181 页）的 setbuf、setvbuf 和图 5.1。

06 UNIX 时间的数据类型

2023-06

题目 UNIX 内核提供的时间以数据类型 time_t 表示。

解析 教材介绍的内核基本时间服务是日历时间，秒数用 time_t 表示；进程 CPU 时间的传统计数则用 clock_t 表示。

题干没有说明时间种类，因此需要按所在章节语境理解，不应从这个判断推出“UNIX 所有时间都用 time_t”。

建议答案 **按教材“基本时间服务”的日历时间语境，答“是”。完整记忆是日历时间用 time_t，进程 CPU 时间的传统计数用 clock_t；若题干明确指“所有种类的时间”，应答“否”。**

教材定位 - §6.10，第 189 页（PDF 第 223 页）首段：内核基本时间服务、Epoch 和 time_t，是判断题的直接对应段落。
- §1.10，第 20 页（PDF 第 54 页）：日历时间 time_t 与进程时间 clock_t 的区分。

07 目录的读权限与执行权限

2023-07

题目 对于目录的读权限和执行权限，意义相同吗？为什么?

解析 目录的 r 权限允许读取目录项、列出名字，x 权限允许搜索目录、按名字访问其中的对象。知道一个文件名，并不意味着有权列出整个目录；能列出名字，也不意味着能打开这些文件。

访问内部文件时还要满足沿途目录和目标文件本身的相应权限。

建议答案 **不相同。读权限允许列出目录中的文件名；执行权限允许搜索目录，并通过目录访问指定名字的文件。**

教材定位 §4.5，第 100 页（PDF 第 134 页）：第一条规则中直接对比目录的 read permission 与 execute/search permission。

08 创建新文件的目录权限

2023-08

题目 为了在一个目录中创建一个新文件，只需对该目录具有写权限。

解析 写权限允许修改目录项，执行权限允许在该目录中搜索和处理文件名。创建新文件通常需要两者同时具备，不能只看 w。

目录的 r 不是创建一个已知名字的新文件的必要条件。

建议答案 **否。需要父目录的写权限和执行权限，即 w+x。**

教材定位 §4.5，第 100 页（PDF 第 134 页）：靠近页末的创建文件规则，明确要求目录的 write permission 和 execute permission。

09 删除文件需要什么权限

2023-09

题目 为了删除一个现有文件，必须对包含该文件的目录具有写权限和执行权限，而对该文件本身则不需要任何权限。

解析 删除文件名修改的是父目录中的目录项，因此通常要求父目录的 w+x，而不要求目标文件本身有读、写或执行权限。

父目录设置了粘滞位时，文件所有权、目录所有权或相应特权还会影响是否允许删除。

建议答案 **按普通文件权限规则，是。删除文件需要父目录的写权限和执行权限，不要求目标文件本身可读、可写或可执行；粘滞位等机制可能增加限制。**

教材定位
- §4.5，第 100 页（PDF 第 134 页）：靠近页末的删除文件规则，检查所在目录的写、执行权限，不要求目标文件本身可读、可写。
- §4.10，第 109 页（PDF 第 143 页）：目录设置粘滞位时的附加限制。

10 当前工作目录与相对路径

2023-10

题目 每个进程都有一个当前工作目录，此目录是搜索所有相对路径名的起点。

解析 按教材通常的路径解析模型，相对路径从调用进程的当前工作目录出发。当前工作目录是进程属性，可以通过 chdir 或 fchdir 改变。

不能把当前工作目录等同于用户主目录，也不能把它等同于可执行文件所在目录。

严格讨论接口时，openat 等 at 系列函数可以显式指定目录描述符作为相对路径的起点；本题按教材对通常路径解析的概括作答。

建议答案 **按教材通常路径解析的语境，是。相对路径通常以进程的当前工作目录为起点。**

教材定位
- §1.4，第 8 页（PDF 第 42 页）“Working Directory”：题干的直接对应段落。
- §4.23，第 135 页（PDF 第 169 页）：当前工作目录是进程属性，chdir、fchdir 可改变它。
- §3.3，第 64-65 页（PDF 第 98-99 页）：openat 对相对路径解析起点的例外，支持解析中的补充说明。

11 不带缓冲的 I/O

2023-11

题目 什么是不带缓冲的 I/O?

解析 这个术语在材料中可能对应两种语境。

第三章把 read、write 等文件描述符接口称为不带缓冲 I/O，强调每次读写都请求内核服务，不经过标准 I/O 库流缓冲。

第五章讨论标准 I/O 的三种缓冲时，“不带缓冲”表示标准 I/O 库不在该流上积攒字符。原整理稿给的是这个定义，不能据此说第三章的定义不成立。

两者都不意味着内核没有缓存。

建议答案 **一般文件 I/O 语境下：指 read、write 等每次都请求内核服务、不经过标准 I/O 库流缓冲的 I/O。若题目承接标准 I/O 的三种缓冲，则答：标准 I/O 库不对该流的字符进行缓冲存储。**

教材定位
- §3.1，第 61 页（PDF 第 95 页）：第三章“不带缓冲 I/O”的定义，每次 read 或 write 都调用内核中的系统调用。
- §5.4，第 146 页（PDF 第 180 页）第 3 项“Unbuffered”：标准 I/O 库不缓冲字符，这是另一种语境。
- §3.13，第 81 页（PDF 第 115 页）首段：内核的 buffer cache/page cache，说明“不带缓冲”不等于内核没有缓存。

12 时钟时间

2023-12

题目 UNIX 中，时钟时间是什么?

解析 时钟时间即墙上时钟时间，是进程运行经过的实际时间，包含执行和等待。系统中其他进程占用 CPU、I/O 等待以及睡眠都会影响这一时间。

用户 CPU 时间是执行用户指令所耗的 CPU 时间，系统 CPU 时间是内核代表进程执行所耗的 CPU 时间，不能与墙钟时间混为一谈。

建议答案 **时钟时间又称墙上时钟时间，是进程运行经过的实际时间，包括执行和等待，其值与系统负载等因素有关。**

教材定位 §1.10，第 20 页（PDF 第 54 页）：三种进程执行时间及 wall clock time 的定义；同页 time 命令示例列出 real、user、sys。

13 修改密码涉及的文件

2023-13

题目 UNIX 中修改密码需要访问到哪些文件?

解析 在常见的本地 shadow 口令机制中，/etc/passwd 提供账号基本信息，/etc/shadow 保存口令哈希和口令老化信息。

修改密码主要更新受保护的口令记录，而不是把明文密码直接写进 /etc/passwd。访问两个文件也不等于每次都要改写两个文件。

建议答案 **主要涉及 /etc/passwd 和 /etc/shadow。前者保存用户基本信息，后者在 shadow 机制中保存口令哈希与老化信息，修改密码主要更新相应的 shadow 口令记录。**

教材定位
- §4.4，第 99 页（PDF 第 133 页）：passwd 程序修改口令文件，直接列出 /etc/passwd 和 /etc/shadow。
- §6.2，第 178 页（PDF 第 212 页），图 6.1 及其下方说明：/etc/passwd 的字段、示例及口令占位符。
- §6.3，第 181 页（PDF 第 215 页），图 6.3 及正文：shadow 口令文件、单向处理后的口令记录与老化字段。

14 绝对路径与相对路径

2023-14

题目 绝对路径和相对路径有什么区别?

解析 /home/student/a.txt 从根目录开始，是绝对路径。a.txt、./a.txt、../a.txt 都不以 / 开头，通常从当前工作目录开始解析，是相对路径。

判断标准是是否以 / 开头，不是路径中是否出现斜线，也不是是否以点开头。

建议答案 **绝对路径以 / 开头，从根目录开始解析；相对路径不以 / 开头，通常从进程当前工作目录开始解析。**

教材定位
- §1.4，第 5 页（PDF 第 39 页）“Pathname”：绝对路径以 / 开头，相对路径的定义。
- §1.4，第 8 页（PDF 第 42 页）“Working Directory”：相对路径与进程当前工作目录的关系。

15 点与点点

2023-15

题目 .（称为点）和 ..（称为点一点）分别指什么?

解析 . 指目录自身，.. 指父目录。根目录中的 .. 仍指根目录。

目录递归程序若不跳过这两个目录项，就可能反复回到当前目录或上一级目录。

建议答案 **. 表示当前目录自身，.. 表示父目录；根目录中的 .. 与 . 都指根目录。**

教材定位 §1.4，第 4 页（PDF 第 38 页）最末段：.、.. 的含义，以及根目录中的 .. 与 . 相同。

16 有访问权限就能读取目录吗

2023-16

题目 对某个目录具有访问权限的任意用户都可以读取该目录。

解析 教材对应句子里的许可是“读该目录的权限”。如果把“访问权限”理解成读权限，这句话成立；如果理解成任意一种许可，尤其只有 x 搜索权限，则不成立。

本题与“目录的读权限和执行权限不同”并不矛盾，区别在于题干把具体权限省略了。

建议答案 **按教材这里的语境，即具有读目录的权限，答“是”；只有 x 搜索权限并不足以读取目录项。**

教材定位
- §4.22，第 130 页（PDF 第 164 页）首段：原文限定为有权限读取该目录，不是仅有搜索权限。
- §4.5，第 100 页（PDF 第 134 页）：目录的读权限与执行/搜索权限并不相同。

17 ls 所需的目录权限

2023-17

题目 若我们想在某个目录下执行 ls 命令，需要对这个目录有什么权限?

解析 如果只要求列出目录中的文件名，关键是 r 权限。路径中沿途目录还要允许搜索。

如果需要 cd 进入目录，或使用 ls -l 取得内部文件的属性，通常还需要该目录的 x 权限。文件名列表与文件属性不是同一层信息。

建议答案 **仅列出文件名需要目录的读权限 r；若还要进入目录或取得内部文件属性，通常还需要执行权限 x。**

教材定位
- §4.5，第 100 页（PDF 第 134 页）：读权限允许列出文件名；执行/搜索权限用于路径查找。
- §4.22，第 130 页（PDF 第 164 页）：读目录所需的权限。进入目录另见 §4.23，第 135 页（PDF 第 169 页）的 chdir。

18 Shell 的定义

2023-18

题目 Shell 是什么?

解析 Shell 读取、解释并执行命令。命令可以来自交互终端，也可以来自脚本文件。Shell 是用户程序，不是内核；命令既可能是外部程序，也可能是 Shell 内建命令。

建议答案 **Shell 是命令行解释器，读取、解释并执行用户命令；输入可以来自终端，也可以来自 Shell 脚本文件。**

教材定位
- §1.3，第 3 页（PDF 第 37 页）“Shells”首段：Shell 的定义，以及终端、脚本两种输入来源。
- §4.23，第 135-136 页（PDF 第 169-170 页），图 4.23 及其后说明：外部程序不能改变父 Shell 的工作目录，cd 因而是内建命令。

19 空洞与磁盘存储区

2023-19

题目 空洞文件需要在磁盘上占用存储区。

解析 现有 2023 年整理稿写“是”，2025 年整理稿写“否”，不能把两份整理稿的判断直接当作统一标准答案。

教材的准确结论是：空洞部分不要求有对应的磁盘存储区，但空洞文件的元数据与已写入部分仍可能占用空间。小空洞还可能与实际数据共用一个已分配块，未必节省空间。

建议答案 **若按本题通常考查的“空洞部分是否必须占用实际存储区”，答“否”。空洞部分不要求分配对应的磁盘数据块，读出时得到值为 0 的字节。**

若题干强调的是“整个空洞文件是否占用任何磁盘空间”，则可能占用，不能答“整个文件完全不占空间”。本题需要明确这一措辞区别。

教材定位

- §3.6，第 68 页（PDF 第 102 页）：lseek 不执行 I/O、越过文件尾再写入可形成空洞、未写入位置读回 0，以及空洞不要求有磁盘存储区的直接原文。
- §3.6，第 68-69 页（PDF 第 102-103 页），图 3.2 及输出：空洞文件示例。
- §4.12，第 111-112 页（PDF 第 145-146 页）“Holes in a File”：逻辑长度与实际磁盘占用的对比。

补拟代码题

以下代码与小问全部为补拟。C01-C05 对应回忆稿记下的五类考点；C06 是额外练习，不代表回忆缺失的第六道原题。

每道题均给出完整程序。默认在 Linux/POSIX 环境中运行，除 C04 故意讨论权限失败外，题设中使用的文件、目录与权限均满足要求；若题目没有另说，按所述成功路径分析，不考虑信号中断。示例中部分错误检查为突出考点而省略，不作为生产代码模板。

C01 读取目录并统计目录项

C01

来源说明 根据“读目录，类似教材图 1.3”的考点补拟，不是原卷代码。

题目

sample 目录中有普通文件 a.txt、b.txt 和子目录 sub，没有其他用户创建的目录项。假设本次 readdir 依次返回 .、..、b.txt、sub、a.txt。

```c
#include <dirent.h>
#include <stdio.h>
#include <string.h>

int main(void)
{
    DIR *dp = opendir("sample");
    struct dirent *entry;
    int count = 0;

    if (dp == NULL)
        return 1;
    while ((entry = readdir(dp)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0)
            continue;
        printf("%s\n", entry->d_name);
        count++;
    }
    closedir(dp);
    printf("count=%d\n", count);
    return 0;
}
```

回答：

1. 程序做什么，输出是什么？
2. count 是普通文件数吗？
3. 输出是否必然按字母顺序？若改成递归遍历，为什么要跳过 . 和 ..？
4. entry->d_name 是完整路径吗？若要取得该项属性，应怎样处理？

解析

opendir 打开目录并返回 DIR 指针；readdir 每次返回一个目录项。程序主动跳过 . 和 ..，其余每个目录项打印一次并计数，包括 sub 这个子目录。

题目给出了本次返回顺序，因此可以确定本题输出。一般情况下，readdir 不承诺按字母顺序返回，不能因为 ls 经常排序就认为 readdir 也排序。

如果扩展成递归程序，. 会把遍历带回自身，.. 会把遍历带回父目录，可能导致重复或循环。本程序本身没有递归，不会因为没跳过这两项就自动发生递归循环。

d_name 只是该目录项的名字，不是完整路径。对 sample 中的项取属性，可以拼接 sample/名字 再调用 stat 或 lstat，也可以通过目录描述符使用相应的 at 接口。

建议答案

输出为：

```
b.txt
sub
a.txt
count=3
```

程序列出并统计除 .、.. 外的目录项，不只统计普通文件。顺序依赖 readdir 的返回顺序；一般没有按字母排序的保证。d_name 是名字，获取属性时必须结合正确的目录位置。

教材定位

- §1.4，第 5 页（PDF 第 39 页），图 1.3：列出目录项的完整示例；第 7 页（PDF 第 41 页）解释返回顺序、DIR、dirent 和 d_name。
- §4.22，第 130-131 页（PDF 第 164-165 页）：目录操作接口、d_name，以及目录项顺序依赖实现。
- §1.4，第 4 页（PDF 第 38 页）最末段：. 与 .. 的含义。

C02 chdir 与 getcwd

C02

来源说明 根据“getcwd 相关代码，类似图 4.24”的考点补拟，不是原卷代码。

题目

Shell 的当前工作目录为 /home/student/work，其中 sub 是一个实际子目录，不是符号链接。程序作为普通外部程序启动，所有 chdir、getcwd 调用均成功，路径长度小于 256 字节。

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    char buf[256];

    chdir("sub");
    getcwd(buf, sizeof(buf));
    printf("first=%s\n", buf);
    chdir("..");
    getcwd(buf, sizeof(buf));
    printf("second=%s\n", buf);
    return 0;
}
```

回答：

1. 程序输出什么？
2. 程序中的相对路径从哪里开始解析？
3. 程序退出后，启动它的 Shell 的当前工作目录是否改变？
4. 为什么 cd 通常要做成 Shell 内建命令？

解析

新程序通常继承启动进程的当前工作目录，所以第一条 chdir 的 sub 相对于 /home/student/work。

第一次 chdir 后，程序进入 /home/student/work/sub。随后 .. 相对于新的当前目录，回到 /home/student/work。getcwd 返回的是当前工作目录的绝对路径。

当前工作目录属于进程。这个外部程序里的 chdir 不会改变父 Shell 的当前工作目录；并不是因为程序在退出时自动“恢复”了 Shell 的目录，而是父 Shell 的目录从未被它改动。

建议答案

输出为：

```
first=/home/student/work/sub
second=/home/student/work
```

相对路径通常以程序当前工作目录为起点。程序退出后，父 Shell 的目录仍是 /home/student/work。cd 通常必须由 Shell 自身执行，才能改变 Shell 自己的当前工作目录。

教材定位

- §4.23，第 135 页（PDF 第 169 页）：chdir、fchdir 接口与图 4.23。
- §4.23，第 136 页（PDF 第 170 页）：父 Shell 工作目录不变、cd 为内建命令的原因，以及 getcwd 接口和输出示例。
- §4.23，第 137 页（PDF 第 171 页），图 4.24：getcwd 示例程序。注意本页页眉虽写 §4.24，但图 4.24 仍属于 §4.23。

C03 lseek 与空洞文件

C03

来源说明 根据“lseek、创建空洞、read、write”的考点补拟，不是原卷代码。

题目

所有接口均成功，read 一次读到文件中全部 8 个字节，各次 write 均完整写入请求的数据。普通文件原先的内容会被 O_TRUNC 清空。

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    unsigned char buf[8];
    int fd = open("hole.dat", O_RDWR | O_CREAT | O_TRUNC, 0600);
    ssize_t n;

    write(fd, "abc", 3);
    lseek(fd, 6, SEEK_SET);
    write(fd, "XY", 2);
    printf("size=%ld\n", (long)lseek(fd, 0, SEEK_END));
    lseek(fd, 0, SEEK_SET);
    n = read(fd, buf, sizeof(buf));
    printf("n=%zd\n", n);
    for (ssize_t i = 0; i < n; i++)
        printf("%02X%s", (unsigned int)buf[i], i + 1 == n ? "\n" : " ");
    close(fd);
    return 0;
}
```

回答：

1. 最终文件长度是多少？程序输出什么？
2. 空洞位于哪些偏移？读出的是字符 '0'、字节值 0，还是 EOF？
3. lseek(fd, 6, SEEK_SET) 执行后，文件长度是否立刻变成 6？
4. 如果去掉写入 XY 的那条语句，最终文件长度是多少？
5. 根据逻辑长度能否直接计算这个文件实际占用了多少磁盘空间？

解析

第一次写入占据偏移 0、1、2，文件长度为 3，当前偏移为 3。lseek 只把当前偏移改成 6，不写数据，也不直接扩长文件。

随后写入 XY，占据偏移 6、7，文件长度变成 8。中间的偏移 3、4、5 是空洞，读出值为 0 的字节。它们不是字符 '0'，也不是 EOF；读到这些位置仍计入 read 返回的字节数。

操作完成后	当前偏移	文件长度
写入 abc	3	3
定位到偏移 6	6	3
写入 XY	8	8
定位到文件开头	0	8
读取 8 字节	8	8

最后按十六进制打印每个字节，所以值为 0 的字节可以清楚显示出来。若误用 %s 输出这个缓冲区，会把第一个零字节当作字符串结束符，不能据此判断文件只有 abc。

建议答案

输出为：

```
size=8
n=8
61 62 63 00 00 00 58 59
```

空洞位于偏移 3-5，读出值为 0 的字节。单独 lseek 不扩长文件；去掉写入 XY 的语句后，文件长度仍为 3。实际磁盘占用还取决于文件系统、块大小、元数据等，不能直接等同于逻辑长度。

教材定位

- §3.6，第 67 页（PDF 第 101 页）：lseek 接口与 SEEK_SET、SEEK_CUR、SEEK_END；第 68 页（PDF 第 102 页）：lseek 不直接扩长文件、空洞字节读回 0。
- §3.6，第 68-69 页（PDF 第 102-103 页），图 3.2：先定位、后写入以创建空洞的示例。
- §3.7、§3.8，第 71-72 页（PDF 第 105-106 页）：read、write 返回值及偏移变化；§4.12，第 111-112 页（PDF 第 145-146 页）：逻辑长度与磁盘占用。

C04 access 与 open 的身份差异

C04

来源说明 根据“access、open、faccessat，类似图 4.8”的考点补拟，不是原卷代码。AT_EACCESS 的例子按支持该标志的 Linux 环境讨论。

题目

secret.txt 的所有者 UID 为 1002，权限为 0400，只有所有者可读。文件路径沿途目录均可搜索，没有 ACL 等额外机制。

程序运行时实际用户 ID 为 1001，有效用户 ID 为 1002，不具有超级用户特权。可以把它理解成执行了由 UID 1002 拥有、设置了 SUID 的程序后所处的身份状态；题目直接给定这些 ID，不需要实际配置 SUID。

```
#define _GNU_SOURCE
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    if (argc != 2)
        return 1;
    int a = access(argv[1], R_OK);
    int b = faccessat(AT_FDCWD, argv[1], R_OK,
   AT_EACCESS);
    int fd = open(argv[1], O_RDONLY);

    printf("access=%d faccessat=%d open=%s\n",
           a, b, fd >= 0 ? "OK" : "FAIL");
    if (fd >= 0)
        close(fd);
    return 0;
}
```

以 secret.txt 作为唯一命令行参数，回答：

1. 程序输出什么？
2. 三次权限检查分别基于什么身份？
3. 如果把 AT_EACCESS 改成 0，faccessat 的结果是什么？
4. access 成功能否保证随后的 open 一定成功？

解析

access 通常按实际用户、实际组等身份检查。本题的实际用户 1001 不是文件所有者，也没有其他读权限，因此返回 -1。

faccessat 设置 AT_EACCESS 后按有效身份检查，有效用户 1002 就是文件所有者，因此成功返回 0。

open 通常按有效身份进行文件访问检查，本题允许读取，因而打开成功。它返回的是非负文件描述符，不是固定返回 0。

AT_FDCWD 表示相对路径从当前工作目录解析；AT_EACCESS 表示使用有效身份。这两个参数作用不同，不能混淆。

建议答案

输出为：

```
access=-1 faccessat=0 open=OK
```

access 用实际身份，带 AT_EACCESS 的 faccessat 用有效身份，open 通常也按有效身份检查。把 AT_EACCESS 改成 0 后，faccessat 按实际身份检查，在本题中返回 -1。

access 成功不能保证之后的 open 一定成功，因为两次调用之间文件或权限可能变化，检查与使用之间存在时间窗口；在特殊身份程序中，还要注意两者检查的身份本来就可能不同。

教材定位

- §4.4，第 98-99 页（PDF 第 132-133 页），图 4.5：实际身份、有效身份与 SUID。
- §4.7，第 102 页（PDF 第 136 页）：access、faccessat 的接口与实际身份检查，以及 AT_FDCWD 的路径含义。
- §4.7，第 103 页（PDF 第 137 页）首段：AT_EACCESS 使用有效身份；同页图 4.8 及输出直接对比 access 与 open。第 104 页（PDF 第 138 页）首段解释二者结果为何不同。
- §3.3，第 65 页（PDF 第 99 页）：检查与使用之间的时间窗口（TOCTTOU），支持最后一问。

C05 read 与 write 的复制循环

C05

来源说明 根据“将标准输入复制到标准输出，类似图 1.4”的考点补拟，不是原卷代码。

题目

标准输入依次提供六个字节 ABCDEF，不包含换行。为便于跟踪，假设三次 read 的返回值依次为 4、2、0，各次 write 均按请求长度完整成功。

```
#include <unistd.h>

int main(void)
{
    char buf[4];
    ssize_t n;

    while ((n = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        ssize_t done = 0;
        while (done < n) {
            ssize_t k = write(STDOUT_FILENO, buf + done, n - done);
            if (k <= 0)
                return 1;
            done += k;
        }
    }
    return n < 0;
}
```

回答：

1. 程序输出什么？有没有额外换行？
2. 按题设，read、write 分别调用几次？
3. 第二次 read 后，buf 的四个字节分别是什么？
4. 如果把内层循环替换为 write(STDOUT_FILENO, buf, sizeof(buf))，结果有什么错误？
5. 内层循环为什么用 done？

解析

第一次 read 读取 ABCD，n=4，写出 4 字节。第二次 read 只把新数据 EF 写到 buf 的前两个位置，后两个位置仍保留上一次的 C、D；本轮有效数据长度只有 2。

第三次 read 返回 0，表示已到输入末尾，不再进入循环。程序没有自行添加换行。

轮次	read 返回值	buf 中的字节	本轮正确写出的数据
1	4	A B C D	ABCD
2	2	E F C D	EF
3	0	不产生新数据	无

read 不会自动在数据后补字符串结束符，也不会清空没有覆盖的缓冲区部分。复制程序不能把它当作总会返回字符串的接口。

write 可能只写出部分请求数据。done 记录本轮已经写出的字节数，后续从 buf+done 继续写剩余部分。本例遇到失败直接退出，不实现信号中断重试。

建议答案

正确输出六个字节 ABCDEF，没有额外换行。按题设，read 调用 3 次，write 调用 2 次。第二次 read 后 buf 为 E、F、C、D。

如果每轮都写 sizeof(buf) 即 4 字节，第二轮会把残留的 CD 也写出，错误输出为 ABCDEFCD。内层循环用于处理短写，确保本轮 n 个有效字节全部写出。

教材定位

- §1.5，第 9 页（PDF 第 43 页），图 1.4：标准输入复制到标准输出，按 read 的返回值调用 write。
- §3.7，第 71 页（PDF 第 105 页）：短读、EOF 和返回字节数；§3.8，第 72 页（PDF 第 106 页）：write 的返回值。
- §3.9，第 72 页（PDF 第 106 页），图 3.5：同类复制程序。done 循环是本补拟题用于处理短写的写法，不是图 1.4 或图 3.5 的原代码。

C06 dup 与独立 open 的偏移量

C06

来源说明 额外补拟练习，用于巩固文件共享与偏移量。不对应已知原题，也不声称是回忆缺失的第六道代码题。

题目

以下所有调用均成功，read、write 均按请求长度完整完成。

```c
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    char buf[2];
    int fd = open("data.txt", O_RDWR | O_CREAT
   | O_TRUNC, 0600);

    write(fd, "ABCDE", 5);
    lseek(fd, 0, SEEK_SET);
    int same = dup(fd);
    int other = open("data.txt", O_RDONLY);

    read(fd, buf, 2);
    fwrite(buf, 1, 2, stdout);
    read(same, buf, 2);
    fwrite(buf, 1, 2, stdout);
    read(other, buf, 2);
    fwrite(buf, 1, 2, stdout);
    printf("\n");
    lseek(same, 0, SEEK_SET);
    read(fd, buf, 1);
    printf("%c\n", buf[0]);
    close(other);
    close(same);
    close(fd);
    return 0;
}
```

回答：

1. 程序输出什么？
2. same 与 fd 是否共享偏移量？other 呢？
3. 为什么对 same 调用 lseek，会影响随后 fd 的 read？
4. close(same) 是否自动关闭 fd？

解析

dup 返回另一个文件描述符，指向同一个打开文件描述，因此 same 与 fd 共享当前偏移量。重新 open 同一文件则得到独立的打开文件描述，本题中的 other 拥有自己的偏移量。

第一次通过 fd 读出 AB，共享偏移量变为 2。随后通过 same 读出 CD，共享偏移量变为 4。other 仍从自己的偏移 0 开始，读出 AB。

对 same 定位到 0 后，共享偏移量被重置，所以通过 fd 再读一个字节得到 A。

操作完成后	fd 与 same 的共享偏移	other 的独立偏移
dup 和第二次 open 完成	0	0
通过 fd 读取 2 字节	2	0
通过 same 读取 2 字节	4	0
通过 other 读取 2 字节	4	2
通过 same 定位到 0	0	2
通过 fd 读取 1 字节	1	2

建议答案

输出为：

```
ABCDAB
A
```

fd 与 same 共享偏移量，other 不共享。close(same) 只关闭这个描述符，不会自动关闭 fd；只要 fd 仍有效，就仍可通过它访问文件。

教材定位

- §3.10，第 74-77 页（PDF 第 108-111 页）：文件描述符表、文件表与当前偏移；图 3.8 在第 76 页（PDF 第 110 页），展示独立打开时的文件表项与偏移。
- §3.12，第 79 页（PDF 第 113 页）：dup、dup2 接口；第 80 页（PDF 第 114 页），图 3.9 及其下方正文：复制的描述符共享文件表项、文件状态标志和当前偏移。
- §3.5，第 66 页（PDF 第 100 页）：close 关闭指定文件描述符。

检查时重点确认的几处表述

1. 两年的 UID 0 题均按“多个登录名可以映射到 UID 0”答“是”，纠正 2023 年整理稿中的“不可以”。
2. time_t 题按教材日历时间语境答“是”，同时保留 clock_t 的区别，不把所有时间混为一谈。
3. “有访问权限就能读目录”中的访问权限按教材的读目录权限理解；只有 x 的解释不成立。
4. 两年的空洞题保留原题，但区分“空洞部分不要求分配存储区”和“整个文件完全不占空间”，不把含混措辞伪装成无条件的标准判断。
5. 2023 年“不带缓冲 I/O”给出第三章接口含义与第五章流缓冲模式含义，避免直接照抄某一种解释后混淆另一个概念。
6. 6 道代码题均为补拟，其中 C01-C05 对应已回忆考点，C06 只是额外练习。

材料与核对依据

- 题目材料：2025 年材料、2023 年材料。其中的现有答案仅作对照，不当作官方答案。
- 教材：Advanced Programming in the UNIX Environment 3rd Edition，正文页码以这份英文第 3 版为准。
- 多个账号共用 UID：useradd 手册，参见 -o/--non-unique 与 -u/--uid。
- 实际身份与有效身份检查：access、faccessat 手册，参见 AT_EACCESS。
- 流的定向与 fwide：fwide 手册。
- cp 命令：GNU Coreutils 文档。